

Midterm Exam

CS 101 Algorithmic Problem Solving L7

Fall 2023

Name(s): _____

HU ID (*e.g., xy01042*): _____

Exam Instructions

- Exam Duration: 2 hours and 30 minutes
- Number of Questions: This exam contains 4 coding questions and 1 debugging question.
- Technical Requirements: You are not allowed to use code editors, compilers or online resources during this exam. Only electronic calculators are permitted during this exam.
- Communication: During the exam, do not communicate with anyone except the instructor or exam proctors if you have questions about the exam.
- Honesty: Academic honesty is expected. Do not engage in any form of cheating, plagiarism, or unauthorized collaboration.
- Answer Format: Type your answers clearly in the provided spaces.
- Time Management: Monitor the time carefully and pace yourself. Do not spend too much time on any one question or section.
- Exam Completion: Before submitting, take a moment to review your answers to ensure you have completed all questions and that your responses are clear and accurate. **Good luck!**

1. Divisible by 3 or not?

Yahya likes the number 3 a lot, he has two non-negative integers A and B .

In one operation he can do either **one** of the following operations:

- $A := |A - B|$ (change A to $|A - B|$)
- $B := |A - B|$ (change B to $|A - B|$)

$|A|$ is used for the absolute value, example: $|-7| = 7$ and $|7| = 7$.

Your job is to write a function `minimum` that determines the minimum number of operations after which at least one integer out of A and B becomes divisible by 3.

Make sure to call your function and to present the output using: `print(functionname(parameters))`

Constraints

- $0 \leq A, B \leq 1000$

Interaction

The input comprises of two space separated integers denoting the value of A and B

The output must contain an integer which is the minimum number of operations after which atleast one integer out of A and B becomes divisible by 3.

Sample

Input	Output
0 343	0
2 7	2

In the first case, $(A, B) = (0, 343)$, $A = 0$ means that it is already divisible by 3. No operations needed hence the output is 0.

In the second case, $(A, B) = (2, 7)$, you can change $B = 7$ to $B = |A - B| = |2 - 7| = 5$, $B = 5$ is still not divisible by 3. Now you change $B = 5$ to $B = |A - B| = |2 - 5| = 3$. B is now divisible by 3 and it took you 2 operations to reach this point hence the output is 2.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
1 1	1
33 31	0
7 14	2

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: A, B

Output: Minimum number of operations needed to make either A or B divisible by 3.

Pseudocode

```
def minimum(A,B):  
    count = 0  
    if A%3 == 0 or B%3 == 0:  
        return(0)  
    if A == B:  
        return count + 1  
  
    while (A%3 != 0) and (B%3 !=0):  
        dif = abs(A-B)  
        if (A>B):  
            A = dif  
            count += 1  
        elif B>A:  
            B = dif  
            count += 1  
        else:  
            return count  
    return count  
print(minimum(A,B))
```

Dry Run

Using any **one** of the inputs provided in the **Exercise section** above, dry run your pseudocode in the space below.

Input: A,B = 7,14

count = 1

if A==B # false

if A%3 == 0 or B%3 == 0 # false

Goes in while loop – while condition true

dif = abs(A-B) = abs(7-14) = 7

Since B>A second elif true

B = 7 and count = 2

loop continues

dif = abs(7-7) = 0 Since A == B last else condition true and return count = 2 which is the expected output!

2. Professor Raheela's Grading Policy

Professor Raheela has the following grading policy:

- Every student receives a grade in the inclusive range from 0 to 100.
- If the difference between the grade and the next multiple of 10 is less than 5, round the grade up to the next multiple of 10.
- If the value of the grade is less than 33, it is considered a failing grade, and no rounding occurs.

Your task is to design a function called **round_grade(g)** that takes a student's grade **g** as input and returns the final grade after rounding as per the Professor's rules. Make sure to call your function and to present the output using:

```
print(functionname(parameters))
```

Constraints

- $0 \leq g \leq 100$

Interaction

The input comprises a single line containing an integer denoting the value of g .

The output must contain a single number denoting final grade after rounding.

Sample

Input	Output
76	80
33	33

In the first case, $g = 76$ and the next multiple of 10 is 80, $80 - 76 = 4$ which is less than 5. Therefore, it is rounded to the next multiple of 10 which is 80.

In the second case, $g = 33$ and the next multiple of 10 is 40, $40 - 33 = 7$ which is not less than 5. Therefore, it is not rounded and remains as is.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
29	29
60	60
45	45

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: g

Output: The final grade based on calculating whether grade can be rounded off or not as per the policy of the Professor.

Pseudocode

```
def round_grade(g):
    # If the grade is less than 33, it's a failing grade and should not be rounded

    if grade < 33:
        return grade
    # Calculate the remainder when dividing the grade by 10
    remainder = grade % 10
    if remainder > 5:
        return grade + (10 - remainder)
    else:
        return grade # No rounding needed
print(round_grade(g))
```

Dry Run

Using any **one** of the inputs provided in the **Exercise section** above, dry run your pseudocode in the space below.

Input: $g = 60$

grade less than 33?	remainder	remainder greater than 5?	return value
No	$60 \% 10 = 0$	False	60

which is the expected output!

3. Collatz's Conjecture

In 1937, Lothar Collatz, a German mathematician, put forth the following conjecture: if you start with any positive integer, and you half the integer if it is even, or triple it and subtract one from it if it is odd, and you keep doing that, you will eventually arrive at one. This conjecture has, to date, not been proven to be true or false. However, it has been verified up to a very large integer.

Write an iterative function named `verify_collatz`, which takes an argument `n` of type `int`, that prints the sequence of numbers generated by the following algorithm:

1. Print the number `n`.
2. If `n` is either 1, 5, or 17, stop.
3. If `n` is even, divide it by 2; otherwise, triple `n` and subtract one from it.
4. Go back to step 1.

Make sure to call your function and to present the output using:

```
print(functionname(parameters))
```

```
# Hint: Use a while loop to continue generating numbers until a stopping
#       condition is met.
while True:
    # Rest of your code...
```

Constraints

- `isinstance(n, int)`
- `n > 0`

Interaction

The input comprise an integer n . The output is the printed sequence of numbers that follow the rules of Collatz's Conjecture.

Sample

Input	Output
1	1
3	3, 8, 4, 2, 1

In the first case $n = 1$. According to the rules, the number is printed to output and since $n = 1$, no further computations are done.

In the second case, $n = 3$, it is printed to output. Since n is not even, it is tripled and 1 is subtracted from it i.e. $3 \times 3 = 9 - 1 = 8$. The next number is 8 which is printed to output. Since 8 is even, we divide it by 2. $8/2 = 4$. Now 4 is printed to output. Since 4 is even, the next number is $4/2 = 2$. 2 is printed to output. Since 2 is even, the next number $= 2/2 = 1$. 1 is printed to output and the program stops. The final output therefore is: 3, 8, 4, 2, 1.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
8	8,4,2,1
14	14,7,20,10,5
17	17

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: n Output: Sequence of numbers generated via Collatz conjecture rules.

Pseudocode

```
def verify_collatz(n):
    while True:
        print(n, end=", ")
        if n == 1 or n == 5 or n == 17:
            break
        elif n % 2 == 0:
            n = n // 2
        else:
            n = (n * 3) - 1

# Example usage:
verify_collatz(n)
```

Dry Run

Using any **one** of the inputs provided in the **Exercise section** above, dry run your pseudocode in the space below.

Input: $n = 8$

is $n = 1$ or 5 or 17?	$n \% 2 = 0$?	n	printed to output
No	True	$n = n // 2 = 4$	8,
No	True	$n = n // 2 = 2$	8,4,
No	True	$n = n // 2 = 1$	8,4,2,
Yes	-	-	8,4,2,1

which is the expected output!

4. The Great Spelling Bee Mix-Up

In the quirky town of Alphaberville, there was an annual event that everyone looked forward to – the Great Spelling Bee Championship. This year, a young contestant named Benny had an uncanny talent for spelling words, but there was just one problem – he had a peculiar way of mixing up letters. When given the word “potato” Benny would spell it as “otatop.” The crowd was both baffled and amused. As the spelling bee progressed, Benny’s unconventional spelling style became the talk of the town. The word “banana” became “anaban” and “computer” became “utercomp,”. Later it was realized that his spelling had a pattern. He was rotating letters.

Given a word, a rotation is found by moving one letter from the beginning to the end of the word. For example, the word ‘orange’ rotated once is ‘rangeo’, rotated twice is ‘angeor’, rotated thrice is ‘ngeora’. Design a function named **rotate** that takes a string **s** and an integer **n** as input and returns the string rotated **n** times.

Make sure to call your function and to present the output using:

```
print(functionname(parameters))
```

Constraints

- `isinstance(n, int)`
- `len(s) > 1`
- `n <= len(s)`

Interaction

The two space separated inputs comprise a string `s` and an integer `n`. The output is the rotated string.

Sample

Input	Output
potato 1	otatop
potato 3	atopot
potato -2	topota

In the first case, since `n = 1`, the string is rotated only once. The first letter 'p' moves to the end, the rest of the string remains the same and hence the output is otatop.

In the second case, since `n = 3`, the first three letters of the string move to the end of the string in the following order: First, p, then o and then t. The rest of the string remains the same and the output is atopot.

In the third case, the negative `n` indicates that the last letters need to move to the beginning of the string. So first o moves to the beginning and then t. Finally the output is topota.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
wonderful -4	rfulwonde
abracadabra 3	acadabraabr
rotor 5	rotor

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: `s,n`

Output: String `s` with `n` letters moved to the beginning from end or vice versa depending on whether `n` is positive or negative.

Pseudocode

```
def rotate(s, n):  
    rotated = s[n:] + s[:n] # Rotate the string  
    return rotated  
  
print(rotate(s,n))
```

Dry Run

Using any **one** of the inputs provided in the **Exercise section** above, dry run your psuedocode in the space below.

w	o	n	d	e	r	f	u	l
-9	-8	-7	-6	-5	-4	-3	-2	-1

```

Input s,n = wonderful, -4
s[n:] = s[-4:] = rful
s[:n] = s[: -4] = wonde
rotated = s[-4:] + s[: -4] = "rful" + "wonde"
rotated = rfulwonde
which is the expected output!

```

5. The Tale of Professor Pascal

In a picturesque town surrounded by rolling hills, Professor Pascal was a beloved figure known for his deep love of mathematics. He received a special invitation to the "Mathematical Marvels" event in a neighboring town, where he was tasked with presenting something truly captivating.

Determined to leave a lasting impression, Professor Pascal turned to the beauty of Pascal's Triangle, a mesmerizing pattern of numbers that held the secrets of mathematics. He set out to create a Python function that would not only explain the concept but would vividly demonstrate its enchanting properties.

This function, which he affectionately named the `pascal_triangle_printer`, could take any positive integer `n` as input and print the first `n` rows of Pascal's Triangle. The triangle was an array of numbers, where each number was the sum of the two numbers directly above it. Professor Pascal believed that this simple rule revealed the magic of mathematics in its purest form.

Constraints

- `isinstance(n, int)`
- `1 <= n <= 100`

Interaction

The input consists of an integer representing `n`. The output is the printed Pascal's triangle.

Sample

Input	Output
3	1 1 1 1 2 1
5	1 1 1 1 2 1 1 3 3 1 1 4 6 4 1

In the first case, `n=3` so there are three rows to display. Each row begins with a 1 and ends with a 1. The middle number is the sum of the two numbers above it (one directly above it, let's call it `m` and the other to the left of `m`).

In the second case, the logic is similar, only number of rows is 5 since $n = 5$.

Proposed Solution

```

1 def print_pascal_triangle(n):
2     for i in range(1,n):
3         current = 1
4         for j in range(1, i + 2):
5             print(current, end=" ")
6             current = current + (i - j + 1) // j
7         print(1)
8
9 print_pascal_triangle(5)

```

Dry Run

Using the input provided in the Proposed Solution above ($n = 5$), dry run the **proposed** code solution below.

$n = 5$

Outer loop (variable i) runs from 1 to 5. So $i = 1, 2, 3, 4$

Inner loop (variable j) runs from 1 to $(i+2)$ so 1 to 3, 1 to 4, 1 to 5, and 1 to 6.

$i = 1$	$i = 2$	$i = 3$
$j = 1, 2$	$j = 1, 2, 3$	$j = 1, 2, 3, 4$
$current = 1$	$current = 1$	$current = 1$
$current = current + (i-j+1)//j$	$(j=1)=1+(2-1+1)//1=1+2=3$	$(j=1)=1+(3-1+1)//1=1+3=4$
$(j=1) = 1+(1-1+1)//1=1+1=2$	$(j=2)=3+(2-2+1)//2=3+0=3$	$(j=2)=4+(3-2+1)//2=4+1=5$
$(j=2) = 2+(1-2+1)//2=2+0=2$	$(j=3)=3+(2-3+1)//3=3+0=3$	$(j=3)=5+(3-3+1)//3=5+0=5$
$print(1)$	$print(1)$	$(j=4)=5+(3-4+1)//4=5+0=5$
		$print(1)$
Output: 1 2 1	1 2 1	1 2 1
	1 3 3 1	1 3 3 1
		1 4 5 5 1

$i = 4$	
$j = 1, 2, 3, 4, 5$	
$current = 1$	
$(j=1)=1+(4-1+1)//1=1+4=5$	
$(j=2)=5+(4-2+1)//2=5+1=6$	
$(j=3)=6+(4-3+1)//3=6+0=6$	
$(j=4)=6+(4-4+1)//4=6+0=6$	
$(j=5)=6+(4-5+1)//5=6+0=6$ – not printed	
$print(1)$	
Output:	Expected Output
	1
1 2 1	1 1
1 3 3 1	1 2 1
1 4 5 5 1	1 3 3 1
1 5 6 6 6 1	1 4 6 4 1

Error Identification

Briefly explain the error(s) you identified in the proposed code solution. Mention the line number and the errors in each line.

- **Line 2:** Outer loop variable should run n times so it should be `range(n)` instead of `range(1,n)`.
- **Line 4:** Inner loop should run from `range(1,i+1)` because at each iteration it is printing one more value than it should.
- **Line 6:** The formula for value printed should be:
`current = current * (i - j + 1) // j`
Incorrect value is printed with the formula given in proposed solution.

Correct Solution

Rewrite the lines of code you mentioned above with their errors corrected.

```
def print_pascal_triangle(n):  
    for i in range(n):  
        current = 1  
        for j in range(1, i + 1):  
            # Print the current number, separated by a space.  
            print(current, end=" ")  
            current = current * (i - j + 1) // j  
        # Print 1 at the end of each row.  
        print(1)  
# Example usage:  
print_pascal_triangle(5)
```

Rough Work